

Trabalho Prático 01 – Classificação KNN

Aluno: Tobias Maugus Bueno Cougo

1. Metodologia

Foram desenvolvidos dois classificadores KNN aplicados à base Iris (150 amostras, 3 classes, 4 atributos), com divisão treino/teste de 70%/30% estratificada (random_state=42): (i) uma implementação **hardcore**, feita do zero em Python/NumPy (distância Euclidiana + votação majoritária), e (ii) uma implementação utilizando o **KNeighborsClassifier** do Scikit-learn. Ambos foram avaliados para $k = \{1, 3, 5, 7\}$ quanto à taxa de reconhecimento (acurácia), e, para o melhor k , quanto à matriz de confusão, precisão e revocação (médias macro).

2. Resultados

Tabela 1. Acurácia por valor de k (idêntica nas duas implementações).

k	1	3	5	7
Acurácia (Hardcore)	0.9333	0.9556	0.9778	0.9556
Acurácia (Scikit-learn)	0.9333	0.9556	0.9778	0.9556

Tabela 2. Métricas de avaliação para o melhor k ($k = 5$) e tempo médio de predição.

Métrica	KNN Hardcore	KNN Scikit-learn
Acurácia	0.9778	0.9778
Precisão (macro)	0.9792	0.9792
Revocação (macro)	0.9778	0.9778
Tempo médio (predição, $k=1..7$)	≈ 81.0 ms	≈ 7 ms

3. Análise e Conclusão

Em termos de **qualidade da classificação**, as duas implementações obtiveram resultados idênticos para todos os valores de k testados, o que era esperado, já que ambas seguem exatamente a mesma lógica matemática (distância Euclidiana e votação majoritária entre os k vizinhos mais próximos). O melhor desempenho foi obtido com $k = 5$ (97,78% de acurácia), enquanto $k = 1$ apresentou o pior resultado, provavelmente por ser mais sensível a ruído/outliers (overfitting local); valores intermediários de k tendem a suavizar a fronteira de decisão e generalizar melhor nesta base.

Já em termos de **desempenho computacional**, a implementação Scikit-learn foi cerca de 11.5 vezes mais rápida que a implementação hardcore. Isso ocorre porque o Scikit-learn utiliza estruturas de dados otimizadas para busca de vizinhos (ex.: KD-Tree/Ball-Tree) e operações vetorizadas/compiladas em C, enquanto a implementação manual calcula a distância a cada vizinho individualmente em laço Python puro, o que é significativamente menos eficiente.

Conclusão: a implementação do zero cumpriu seu papel didático, comprovando que, do ponto de vista de acurácia/precisão/revocação, ela é equivalente à biblioteca pronta. Contudo, para uso em aplicações reais, especialmente com bases maiores, o Scikit-learn é a opção mais indicada, por sua maior eficiência computacional e recursos adicionais já otimizados.

4. Repositório com código, apresentação e vídeo

Link: <https://github.com/TobiasMaugus/IA-Projects.git>